

Visite Virtuelle Carting Hergla Park

Documentation technique de l'avancement — Semaines 1 & 2

AUTEURS

Équipe de Développement (Rourou & Sassi)

DATE

Juillet 2026

STATUT

Rapport d'Étape Académique

Sommaire

3. Introduction	3
4. Architecture globale du système	3
5. Partie 1 — Le Backend	5
5.1 Choix de l'architecture NestJS + Prisma + PostgreSQL	
5.2 Modélisation et relations de données (User, Espace)	
5.3 Sécurité : Hachage Bcrypt, Tokens JWT et Guards de rôles	
5.4 Documentation de l'API avec Swagger OpenAPI	
6. Partie 2 — Le Dashboard Admin	9
6.1 Objectifs fonctionnels et interface	
6.2 Choix techniques et architecture React	
6.3 Gestion du support bilingue Français/Arabe et inversion RTL	
7. Partie 3 — Le Site "Visite Virtuelle"	11
7.1 Rôle du site vitrine et sas de transition	
7.2 Le parcours d'onboarding en 4 étapes	
7.3 Intégration future du module Unity WebGL	
8. Synthèse des technologies utilisées	13
9. Défis rencontrés et solutions apportées	15
10. Prochaines étapes de développement	15
11. Conclusion	16

3. Introduction

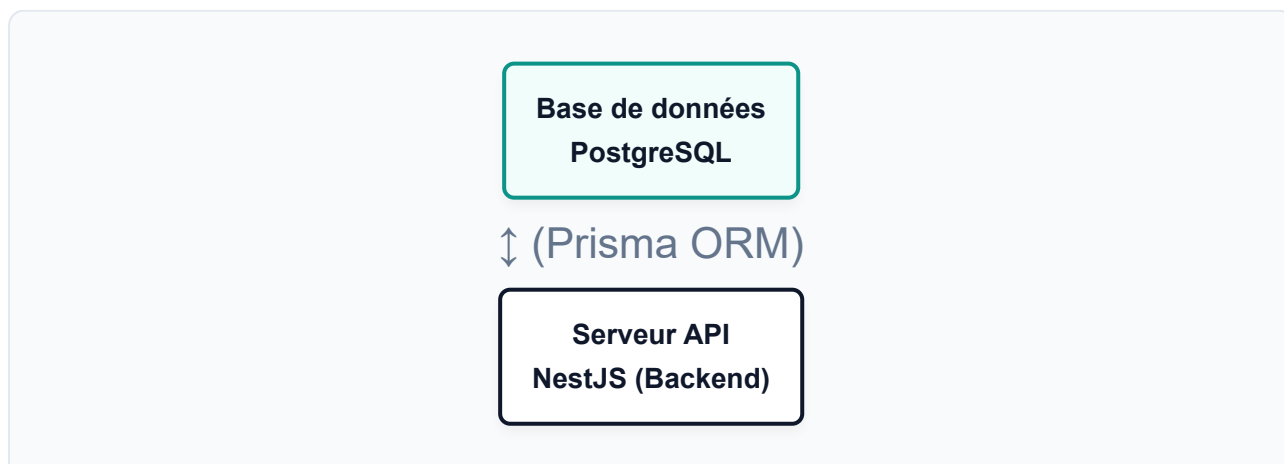
Le projet de **Visite Virtuelle du Carting Hergla Park** s'inscrit dans une démarche de modernisation numérique pour le parc d'attractions de Hergla. L'objectif principal est de concevoir et de déployer une plateforme web interactive permettant aux visiteurs d'explorer le parc à travers une expérience 3D/VR immersive directement depuis leur navigateur web. Au-delà du volet immersif public, le système intègre un outil de gestion administrative permettant aux équipes opérationnelles d'actualiser en temps réel l'état des différents espaces (karting, restaurant, aires de jeux).

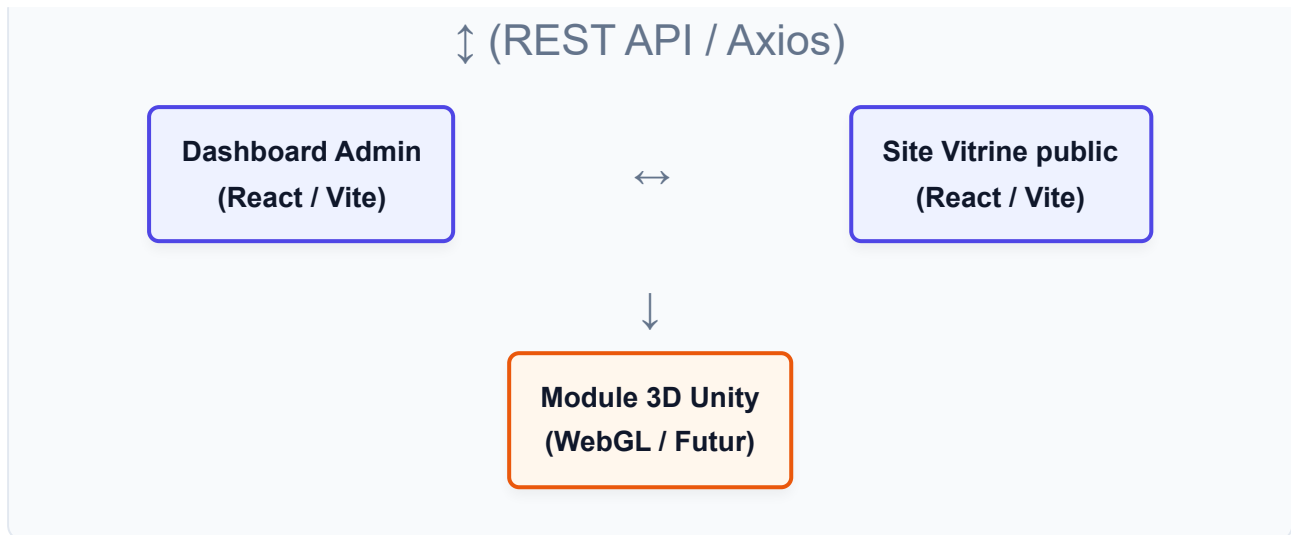
Ce rapport présente l'avancement technique accompli durant les **Semaines 1 & 2**. Cette période a été consacrée à la mise en place des fondations robustes du projet : la conception et le déploiement du backend de l'API, la modélisation de la base de données, la réalisation du Dashboard administratif pour le personnel, et la construction du parcours d'onboarding (sas d'accueil) du site vitrine. Ce document détaille les architectures choisies, justifie les technologies employées et explicite leur fonctionnement dans un langage accessible.

Portée du document : Ce rapport couvre l'intégralité des briques logicielles opérationnelles à ce jour (API NestJS, base de données PostgreSQL, Dashboard d'administration React et Site de Visite Virtuelle React séparé). Les développements liés à la modélisation 3D et au moteur de jeu Unity font l'objet des étapes futures (Semaines 3 à 8).

4. Architecture globale du système

Pour garantir la flexibilité, la scalabilité et l'indépendance des plateformes, le système repose sur une architecture découplée orientée services. Les briques applicatives communiquent entre elles à l'aide de protocoles standards.





Chaque liaison du schéma représente un flux de communication précis du système :

- **Base de données PostgreSQL ↔ API NestJS** : Le serveur API interroge et modifie les tables de la base de données via l'ORM *Prisma* (un outil de traduction entre le code et le langage SQL).
- **API NestJS ↔ Dashboard Admin** : Le personnel se connecte et modifie l'état du parc via des requêtes HTTP (méthode REST) sécurisées par un token JWT (un jeton d'accès temporaire chiffré).
- **API NestJS ↔ Site Vitrine public** : Le site vitrine interroge l'API en lecture seule pour afficher aux visiteurs l'état actuel (ouvert ou fermé) des différentes attractions en temps réel.
- **Site Vitrine ↔ Module Unity WebGL** : Le site public sert de sas de préparation pour l'utilisateur avant d'injecter et de lancer le moteur 3D interactif dans le navigateur.

5. Partie 1 — Le Backend

5.1 Choix de l'architecture NestJS + Prisma + PostgreSQL

Pour développer le cœur logique de l'application (le Backend), nous avons opté pour le framework **NestJS** couplé à l'ORM **Prisma** et à la base de données relationnelle **PostgreSQL**.

- **NestJS (Framework Node.js en TypeScript)** : Il impose une architecture modulaire stricte et robuste (division en contrôleurs pour gérer les routes, services pour la logique métier et modules pour encapsuler les fonctionnalités). Cela évite le "code spaghetti" et assure que l'application reste maintenable à mesure qu'elle grandit.
- **Prisma (ORM - Object-Relational Mapping)** : Il remplace l'écriture manuelle de requêtes SQL complexes par des fonctions écrites directement en TypeScript, garantissant le typage des données. De plus, son système de migrations versionnées conserve un historique précis des modifications de la base de données.
- **PostgreSQL (Base de données relationnelle)** : Réputé pour sa fiabilité et sa gestion rigoureuse des intégrités de données. Il est exécuté à l'aide de **Docker Compose**, ce qui permet de lancer la base de données dans un conteneur isolé en une seule commande, sans polluer la machine de développement.

5.2 Modélisation et relations de données (User, Espace)

La base de données repose sur deux entités clés modélisées dans le schéma Prisma : l'utilisateur (`User`) et la zone d'attraction (`Espace`).

Schéma relationnel simplifié (1 à N) :

Un `Espace` (ex: la piste de karting) peut être assigné à plusieurs employés (`User`).

Chaque `User` (s'il est employé ou administrateur d'espace) est rattaché à un unique `Espace` via son identifiant `assignedSpaceId` .

Détails des champs clés de la base de données :

- **User (Utilisateur)** :
 - `id` : Identifiant unique universel (UUID).
 - `nom` & `email` (unique, utilisé pour la connexion).
 - `passwordHash` : Version chiffrée sécurisée du mot de passe.

- `role` : Rôle de l'utilisateur dans le système, défini par une liste restreinte (`SUPERADMIN` , `ADMIN` , ou `EMPLOYE`).
- **Espace (Attraction/Zone du parc) :**
 - `id` : Identifiant unique universel (UUID).
 - `nom` & `categorie` (ex: "karting", "restauration", "jeux").
 - `statut` : État actuel opérationnel de l'attraction (`OUVERT` , `FERME` , ou `MAINTENANCE`).
 - `donneesSpecifiques` : Un champ de format **JSON** flexible. C'est un choix de conception majeur qui permet de stocker des données uniques propres à chaque type d'attraction sans modifier le schéma de la base (ex: le menu du jour pour le restaurant, ou les meilleurs temps au tour pour la piste de karting).

Exemple de données pour la Piste de Karting :

Champ	Valeur type stockée	Rôle / Explication
<code>nom</code>	"Piste de Karting Hergla"	Nom officiel affiché
<code>categorie</code>	"karting"	Catégorie pour le filtrage et les icônes
<code>statut</code>	OUVERT	État actuel visible par le public
<code>donneesSpecifiques</code>	{ "temps_record": "41.87s", "nom_pilote": "Amine S.", "karts_actifs": 12 }	Données flexibles encodées en JSON, éditables par l'employé assigné

5.3 Sécurité : Hachage Bcrypt, Tokens JWT et Guards de rôles

Pour protéger les données sensibles et restreindre l'accès aux interfaces d'administration, le backend implémente une stratégie de sécurité moderne et robuste divisée en deux couches distinctes :

Le hachage de mot de passe (Bcrypt)

Il est interdit de stocker les mots de passe des utilisateurs en texte brut dans la base de données. Pour y remédier, nous utilisons la bibliothèque **Bcrypt**. Lors de l'inscription d'un utilisateur, son mot de passe subit une fonction mathématique à sens unique qui le transforme en une empreinte numérique indéchiffrable (le `passwordHash`). Même si la base de données était compromise, les pirates ne pourraient pas retrouver les mots de passe d'origine.

L'authentification par jeton d'accès (JWT - JSON Web Token)

Pour éviter que l'utilisateur n'envoie ses identifiants à chaque requête HTTP, le backend utilise des jetons **JWT**. On peut comparer le JWT à un **"badge d'accès temporaire"** :

1. L'utilisateur envoie son email et mot de passe au point d'accès de connexion (`POST /api/auth/login`).
2. Le serveur valide les identifiants et génère un jeton chiffré contenant l'identifiant et le rôle de l'utilisateur. Ce jeton a une durée de validité limitée.
3. L'application cliente (Dashboard React) enregistre ce jeton et l'envoie automatiquement dans l'en-tête de chaque requête HTTP (sous la forme `Authorization: Bearer`).
4. Le serveur valide la signature du jeton pour identifier l'expéditeur et autoriser la requête.

Les vérificateurs d'accès (Guards)

NestJS utilise des *Guards* (garde-fous) pour intercepter les requêtes entrantes avant qu'elles n'atteignent le contrôleur :

- **AuthGuard** : Vérifie que le jeton JWT envoyé par le client est présent, valide et non expiré.
- **RolesGuard** : Lit le rôle inscrit dans le JWT et le compare aux rôles requis pour accéder à l'action demandée. Par exemple, si une route demande le rôle `SUPERADMIN`, le garde-fou bloquera immédiatement un utilisateur ayant le rôle `EMPLOYEE` en lui renvoyant une erreur `403 Forbidden` (Accès interdit).

5.4 Documentation de l'API avec Swagger OpenAPI

Afin de faciliter la collaboration et de permettre aux développeurs frontend de tester l'API de manière interactive, nous avons intégré **Swagger** (via `@nestjs/swagger`). Cet outil génère automatiquement une documentation dynamique accessible en développement sur l'URL `/api-docs`.

Chaque route (`/api/auth`, `/api/users`, `/api/espaces`) y est décrite avec les paramètres requis, le format JSON attendu en entrée et les codes de réponse renvoyés par le serveur. L'interface intègre un module d'authentification où l'on peut coller le jeton JWT pour effectuer des tests en conditions réelles directement depuis le navigateur.

Procédure de validation manuelle :

Le bon fonctionnement des restrictions de sécurité a été validé selon le scénario suivant :

1. Enregistrement d'un utilisateur `SUPERADMIN` via l'API d'authentification.
2. Connexion pour récupérer le jeton JWT et configuration de l'autorisation dans Swagger.
3. Création d'un espace de karting avec l'ID généré par la base de données.
4. Création d'un compte `EMPLOYEE` assigné à cet espace spécifique.
5. Tentative de modification d'un autre espace par cet employé : rejet automatique par le `RolesGuard` (erreur 403). Modification réussie de son propre espace : validée avec succès (code 200).

6. Partie 2 — Le Dashboard Admin

6.1 Objectifs fonctionnels et interface

Le Dashboard Admin est l'application interne destinée au personnel du parc Hergla Park. Il offre une interface visuelle simple permettant aux équipes sur le terrain de gérer l'état opérationnel des attractions sans connaissances techniques.

- **Vue SUPERADMIN** : Il dispose d'une grille de synthèse affichant l'état de l'ensemble du parc en temps réel (nombre total d'espaces ouverts, fermés ou en maintenance, nombre d'employés actifs). Il peut créer de nouveaux espaces, gérer et modifier le profil de tous les employés (CRUD complet).
- **Vue ADMIN / EMPLOYE** : Redirigé vers sa page dédiée "Mon Espace", il ne voit que l'attraction à laquelle il est assigné. Il dispose de boutons d'action rapide pour changer le statut (Ouvert / Fermé / Maintenance) et d'un formulaire pour éditer les données spécifiques au format JSON (comme les scores ou temps records).

6.2 Choix techniques et architecture React

Le dashboard a été conçu avec la stack **React 19**, **Vite** et **Tailwind CSS v4**.

- **React + Vite** : React permet de créer une interface réactive où les composants se mettent à jour instantanément sans recharger la page. Vite sert d'outil d'assemblage et offre un serveur de développement ultra-rapide.
- **Axios + Intercepteurs** : Pour communiquer avec le backend, nous utilisons le client HTTP *Axios*. Un intercepteur a été configuré pour injecter automatiquement le jeton JWT (récupéré dans le stockage local) dans toutes les requêtes sortantes. Si le serveur renvoie une erreur d'authentification (code 401), l'intercepteur déconnecte l'utilisateur et le redirige vers la page de connexion.
- **Mise à jour optimiste (Optimistic UI)** : Lors du clic sur le changement d'état d'un espace, l'interface change d'aspect immédiatement avant de recevoir la réponse du serveur. Si la requête réseau réussit, le changement est validé ; si elle échoue, l'interface revient en arrière de manière transparente pour l'utilisateur. Cela procure une sensation de rapidité extrême de l'application.

6.3 Gestion du support bilingue Français/Arabe et inversion RTL

Le personnel de Hergla Park étant local, il est indispensable de proposer une interface utilisable aussi bien en français qu'en arabe. Nous avons utilisé **react-i18next** pour gérer la traduction de tous les textes de l'application.

L'écriture arabe s'effectuant de droite à gauche, le support technique implique d'inverser totalement le sens de lecture de l'interface (mode **RTL - Right to Left**).

Fonctionnement technique simplifié du RTL :

Quand l'utilisateur sélectionne la langue arabe :

1. Le script applique le paramètre `dir="rtl"` sur l'élément racine du site.
2. Le navigateur inverse automatiquement l'alignement du texte et le sens de lecture.
3. Les classes Tailwind logiques (ex: `ms-4` pour *margin-start* ou `pe-6` pour *padding-end*) s'adaptent de manière dynamique, garantissant que les marges à gauche en français se transforment d'elles-mêmes en marges à droite en arabe sans casser la mise en page.

7. Partie 3 — Le Site "Visite Virtuelle"

7.1 Rôle du site vitrine et sas de transition

Contrairement au Dashboard d'administration, le site de la Visite Virtuelle est une interface publique accessible à tous les internautes. Son but est d'accueillir les futurs visiteurs de Hergla Park, de susciter l'intérêt et de les préparer avant le lancement de la simulation 3D interactive (qui peut être lourde à charger et nécessite des instructions d'utilisation). Le site est pensé comme un **sas d'onboarding** progressif, réduisant ainsi les taux d'abandon lors du chargement des ressources 3D.

7.2 Le parcours d'onboarding en 4 étapes

L'expérience utilisateur est structurée sous forme de parcours guidé composé de 4 étapes successives. Ce cheminement s'accompagne d'une transition d'ambiance visuelle progressive : on débute avec des teintes chaleureuses illustrant le parc réel pour glisser graduellement vers une ambiance technologique et immersive (teintes sombres et néons turquoise).

1. **Étape 1 — Bienvenue** : Page d'accueil publique présentant le parc d'attractions avec une galerie d'images dynamiques et une description générale. L'utilisateur clique sur le bouton principal pour initier son voyage virtuel.
2. **Étape 2 — Contrôles** : Présentation pédagogique des touches du clavier et des actions de la souris indispensables pour se déplacer dans l'espace 3D (Z-Q-S-D pour la marche, souris pour l'orientation du regard). Cette étape évite que l'utilisateur ne se sente perdu au démarrage.
3. **Étape 3 — Sécurité** : Sensibilisation aux règles de conduite à tenir au sein du parc virtuel (respect du sens de la piste de karting, zones piétonnes sécurisées).
4. **Étape 4 — Lancer** : Un écran de chargement simulé avec une jauge de progression fluide de 3,5 secondes, débouchant sur l'affichage du module d'interaction final (simulé par un placeholder esthétique).

Toutes les transitions entre les étapes sont animées de manière fluide à l'aide de la bibliothèque **framer-motion**, offrant un rendu haut de gamme et immersif.

7.3 Intégration future du module Unity WebGL

Actuellement, l'étape 4 affiche un composant temporaire (placeholder) simulant la présence du jeu 3D. L'intégration finale du module Unity, planifiée pour la Semaine 8, s'effectuera selon le protocole suivant :

- Le projet Unity sera compilé au format **WebGL** (un standard permettant d'exécuter du contenu 3D dans le navigateur sans plugin additionnel).
- Les fichiers compilés (code WASM, ressources graphiques et chargeur de scripts) seront déposés dans le dossier `public/unity-build/` du projet React.
- Nous installerons la bibliothèque `react-unity-webgl` pour instancier le lecteur Unity officiel sous forme de composant React et le connecter aux étapes d'onboarding.

8. Synthèse des technologies utilisées

Le tableau ci-dessous dresse une synthèse claire de l'ensemble des technologies sélectionnées et déployées durant les deux premières semaines du projet.

Technologie	Rôle dans le projet	Pourquoi ce choix ?
NestJS	Moteur logique de l'API (Backend)	Structure modulaire stricte en TypeScript, idéal pour une API propre et évolutive.
Prisma	ORM (Traducteur Base de Données)	Typage fort éliminant les erreurs SQL, migrations de base de données documentées.
PostgreSQL	Base de données relationnelle	Stabilité, gestion rigoureuse des relations complexes et des données au format JSON.
Docker	Conteneurisation de la base de données	Déploiement rapide de PostgreSQL dans un environnement isolé identique pour tous.
JWT & Bcrypt	Sécurité et Authentification	Hachage fort des mots de passe (Bcrypt) et jetons de session sans état hautement sécurisés (JWT).
Swagger	Documentation interactive de l'API	Génération automatique des spécifications OpenAPI pour faciliter le test des routes HTTP.
React 19 & Vite	Interfaces utilisateurs (Admin & Public)	Performances au rendu élevées, développement rapide et architecture basée sur des composants réutilisables.
Tailwind CSS v4	Mise en page et identité visuelle	Vitesse d'intégration, support natif du mode sombre et des marges logiques pour le RTL.

Technologie	Rôle dans le projet	Pourquoi ce choix ?
react-i18next	Traduction et gestion des langues	Gestion propre des fichiers de traduction et mise à jour dynamique du sens de lecture LTR/RTL.
Axios	Client de requêtes HTTP	Gestion simple des appels API avec intercepteurs pour insérer le badge JWT automatiquement.

9. Défis rencontrés et solutions apportées

Le développement de ces deux premières semaines a fait face à quelques arbitrages et défis techniques notables, résolus avec succès :

- **Changement de la structure de base de données** : Initialement envisagée de manière classique avec de multiples tables figées pour chaque type de données d'attraction, nous avons choisi d'utiliser le type `Json` de PostgreSQL via Prisma pour le champ `donneesSpecifiques` . Cela offre une flexibilité totale sans alourdir le schéma relationnel de la base.
- **Intégration du sens de lecture RTL (Arabe)** : La configuration manuelle des marges et alignements de textes s'est heurtée à des décalages visuels lors de la bascule de langue. L'adoption des classes logiques Tailwind (comme `start` et `end` à la place de `left` et `right`) a permis de standardiser la mise en page de manière robuste.
- **Fluidité du changement de statut** : Pour pallier la latence réseau lors de l'interaction sur l'interface d'administration, l'application utilise des mises à jour optimistes (changement visuel instantané et retour arrière en cas d'erreur de communication de l'API), ce qui évite toute sensation de ralentissement pour le personnel.

10. Prochaines étapes de développement

Le planning de développement se poursuit conformément au calendrier prévisionnel établi sur 8 semaines :

Période	Objectifs techniques	Livrables attendus
Semaines 3 & 4	Modélisation 3D du parc et du circuit	Modèles 3D texturés des karts, des pistes, des bâtiments et du mobilier extérieur du parc.
Semaines 5 & 6	Intégration Unity et moteur physique	Configuration de la physique de conduite du karting, implémentation des caméras et de la navigation.
Semaine 7	Liaison API et synchronisation	Lecture des états réels des espaces depuis le build WebGL et envoi des scores en temps réel au backend.

Période	Objectifs techniques	Livrables attendus
Semaine 8	Optimisations, tests et déploiement	Optimisation du chargement des textures, tests croisés sur navigateurs et mise en ligne finale de la plateforme.

11. Conclusion

Les fondations techniques indispensables au projet de **Visite Virtuelle du Carting Hergla Park** sont désormais entièrement posées et fonctionnelles. La séparation claire entre le serveur d'API (NestJS), le panneau de contrôle de l'administration (React) et le portail de visite virtuelle (React) assure une architecture moderne, fluide et sécurisée. Le respect rigoureux du planning durant ces deux premières semaines de travail garantit le bon déroulement de la suite du projet, axée sur la modélisation et l'intégration interactive de la simulation 3D sous Unity WebGL.